

Physics Embedded Transformers: Module Documentation

Project Documentation

July 18, 2026

Contents

1	Overview	4
2	Repository Layout	4
3	Configuration Layer	4
3.1	Files	4
3.2	Main Function	5
3.3	Behavior	5
3.4	Validation	5
4	Device and Experiment Configuration	5
4.1	Device: <code>asymmetric_loop_h100</code>	5
4.2	Experiment: <code>video_2</code>	6
5	Centerline Geometry Module	6
5.1	Files	6
5.2	Input	7
5.3	Ordering and Orientation	7
5.4	Arc Length	7
5.5	Tangents and Normals	8
5.6	Typed Containers	8
5.7	Outputs	8
5.8	Execution	8
6	Device Region Labeling Module	9
6.1	Files	9
6.2	Purpose	9
6.3	Label IDs	9
6.4	Construction	9
6.5	Validation	10
6.6	Outputs	10
6.7	Execution	10

7	Droplet Fractional Occupancy Module	10
7.1	Files	10
7.2	Inputs	11
7.3	Ellipse Footprint	11
7.4	Raw Occupancy	11
7.5	Normalized Occupancy	12
7.6	Diagnostic Low Coverage	12
7.7	CSV Columns	12
7.8	Outputs	12
7.9	Execution	13
8	Baseline Branch Hydraulics Module	13
8.1	Files	13
8.2	Purpose	13
8.3	Effective Branch Occupancy	13
8.4	Isolated-Droplet Equivalent Length	13
8.5	Effective Branch Lengths	14
8.6	Total Mixture Flow	14
8.7	Parallel Branch Split	14
8.8	Superficial Mixture Velocity	14
8.9	Canonical ML Features	15
8.10	Baseline Assumptions	15
8.11	Outputs	15
8.12	Execution	16
9	Droplet Branch-Velocity Validation Module	16
9.1	Files	16
9.2	Purpose	16
9.3	Measured Velocity Derivation	16
9.4	Unit Interpretation	17
9.5	Normalization	17
9.6	Selection	17
9.7	Selected Track Statistics	18
9.8	Outputs	18
9.9	Execution	18
10	Generated Data Products	18
10.1	Visual Summary	18
10.2	Geometry Products	29
10.3	Device Region Products	30
10.4	Occupancy Products	30
10.5	Hydraulic Products	30
10.6	Branch-Velocity Validation Products	30
11	Testing	30
12	Scope Boundaries	31
13	Recommended Pipeline Order	31

List of Figures

1	Device-level region overlay produced by <code>scripts/build_device_regions.py</code> . The overlay is generated from the precomputed integer region labels, not from visualization-time region inference.	10
2	Centerline geometry diagnostic showing the ordered device centerline representation and derived branch structure.	19
3	Legacy centerline-region diagnostic for frame 0. This plot was useful during development, but the current occupancy pipeline uses the device-level region labels shown in Figure 1.	20
4	Droplet fractional-occupancy diagnostics summarizing coverage, normalized occupancy behavior, and region assignment statistics.	20
5	Representative occupancy spot checks used to visually verify ellipse rasterization against the precomputed region labels.	21
6	Random sample of ten previously invalid or low-coverage occupancy cases, used for manual inspection of boundary and coverage behavior.	22
7	Baseline branch-hydraulics diagnostics showing branch flow-rate and superficial-velocity behavior after normalized branch occupancy is converted into hydraulic resistance.	23
8	Pooled normalized branch-velocity validation scatter. The x-axis is normalized calculated branch superficial velocity in $\mu\text{m/s}$; the y-axis is normalized measured droplet speed derived in <code>px/frame</code> . The plot compares temporal variation after per-track normalization, not absolute velocity magnitudes.	24
9	Selected validation track 3463 in the left branch. The measured droplet-speed signal is derived by centered finite difference from <code>centroid_x/centroid_y</code> ; both signals are shown as per-track z-scores.	25
10	Selected validation track 254 in the left branch. The comparison is zero-lag and unsmoothed.	25
11	Selected validation track 3127 in the left branch.	26
12	Selected validation track 1713 in the left branch.	26
13	Selected validation track 3460 in the left branch.	27
14	Selected validation track 3461 in the right branch.	27
15	Selected validation track 257 in the right branch.	28
16	Selected validation track 3465 in the right branch.	28
17	Selected validation track 3462 in the right branch.	29
18	Selected validation track 1712 in the right branch.	29

1 Overview

This document describes the project modules created so far for the `physics_embedded_transformers` repository. The current implementation builds a deterministic physical preprocessing stack for a microfluidic asymmetric-loop device:

1. Configuration loading.
2. Centerline geometry construction.
3. Device-level physical region labeling.
4. Droplet fractional occupancy.
5. Baseline branch hydraulics.
6. Droplet branch-velocity validation.

The modules are intentionally scoped to deterministic preprocessing and baseline physics. They do not implement training, transformer features, hydraulic parameter fitting, droplet–droplet interaction corrections, or junction resistance models.

2 Repository Layout

Path	Purpose
<code>configs/devices/asymmetric_loop_h100.yml</code>	Device geometry, calibration, region, and hydraulics settings.
<code>configs/experiments/video_2.yml</code>	Experiment metadata, phase flow rates, and input paths.
<code>src/config/loader.py</code>	Experiment-to-device YAML loader.
<code>src/geometry/centerlines.py</code>	Centerline schema inspection, ordering, orientation, geometry.
<code>src/geometry/types.py</code>	Dataclasses for typed centerline and device geometry containers.
<code>src/geometry/regions.py</code>	Device-level integer region label map construction and validation.
<code>src/occupancy/ellipse.py</code>	Local axis-aligned ellipse rasterization.
<code>src/occupancy/calculator.py</code>	Droplet fractional occupancy calculation and summary statistics.
<code>src/hydraulics/baseline.py</code>	Baseline branch-hydraulics equations.
<code>scripts/build_geometry.py</code>	CLI for centerline geometry artifacts.
<code>scripts/build_device_regions.py</code>	CLI for device region labels and overlay.
<code>scripts/compute_droplet_occupancy.py</code>	CLI for droplet occupancy outputs.
<code>scripts/compute_baseline_hydraulics.py</code>	CLI for frame-level baseline hydraulics.
<code>scripts/validate_droplet_branch_velocity.py</code>	Standalone normalized branch-velocity validation analysis.
<code>tests/</code>	Focused unit tests for geometry, regions, occupancy, hydraulics.

3 Configuration Layer

3.1 Files

- `src/config/loader.py`
- `src/config/__init__.py`

3.2 Main Function

```
load_experiment_config(  
    experiment_path: str | Path,  
    configs_root: str | Path = "configs",  
) -> dict
```

3.3 Behavior

The loader reads an experiment YAML file, extracts `experiment.device_id`, resolves the linked device file as:

```
configs/devices/<device_id>.yaml
```

It then loads the device YAML and verifies that:

```
device.id == experiment.device_id
```

The returned structure is:

```
{  
    "experiment": <full experiment YAML mapping>,  
    "device": <full device YAML mapping>  
}
```

3.4 Validation

The loader raises clear exceptions for:

- missing experiment files;
- empty YAML files;
- malformed YAML files;
- missing `experiment.device_id`;
- missing linked device files;
- mismatched experiment and device IDs.

4 Device and Experiment Configuration

4.1 Device: `asymmetric_loop_h100`

The device YAML stores fixed physical geometry and calibration:

- calibration: $\mu\text{m}/\text{px} = 4.0$;
- channel width: $100\ \mu\text{m}$;
- channel height: $100\ \mu\text{m}$;

- left branch length: 1791 μm ;
- right branch length: 1491 μm ;
- junction definitions: 100 $\mu\text{m} \times 100 \mu\text{m}$;
- region artifact paths;
- isolated-droplet hydraulic resistance ratio.

The hydraulics section stores the dissertation provenance:

```
hydraulics:
  isolated_droplet_resistance:
    representation: equivalent_length
    ratio_to_short_branch: 0.15
```

4.2 Experiment: video_2

The experiment YAML stores:

- device link: `device_id: asymmetric_loop_h100`;
- raw video location;
- processed data paths;
- continuous-phase flow rate;
- dispersed-phase flow rate;
- droplet nominal diameter and confinement regime.

For the current experiment:

$$Q_{\text{continuous}} = 1950 \mu\text{L/hr}, \quad Q_{\text{dispersed}} = 10 \mu\text{L/hr}.$$

The total mixture throughput used by baseline hydraulics is:

$$Q_{\text{mixture}} = Q_{\text{continuous}} + Q_{\text{dispersed}} = 1960 \mu\text{L/hr}.$$

5 Centerline Geometry Module

5.1 Files

- `src/geometry/centerlines.py`
- `src/geometry/types.py`
- `scripts/build_geometry.py`
- `tests/test_centerline_geometry.py`

5.2 Input

The geometry builder uses:

`data/processed/2/centerlines.csv`

The discovered schema is:

Column	Meaning
<code>x</code>	centerline x coordinate in pixels
<code>y</code>	centerline y coordinate in pixels
<code>channel</code>	branch label

The expected branch labels are:

`inlet, outlet, left, right`

5.3 Ordering and Orientation

Each branch is represented as an ordered polyline. If explicit ordering columns exist, they are preferred. Otherwise, the module attempts path reconstruction from point adjacency. If adjacency reconstruction is unsuitable for the real CSV, row order is preserved as the implicit sequence and this choice is reported.

Branches are oriented according to the physical flow direction:

- inlet: upstream boundary to upper junction;
- left branch: upper junction to lower junction;
- right branch: upper junction to lower junction;
- outlet: lower junction to downstream boundary.

5.4 Arc Length

Segment length is Euclidean in pixel coordinates:

$$\Delta s_k = \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2}.$$

Cumulative arc length is:

$$s_0 = 0, \quad s_k = \sum_{i=0}^{k-1} \Delta s_i.$$

Physical arc length uses the device calibration:

$$s_{\mu m} = s_{\text{px}} \cdot 4.0.$$

5.5 Tangents and Normals

Interior tangents use centered finite differences. Endpoints use one-sided finite differences. Tangents are normalized to unit length.

The normal convention is:

$$n_x = -t_y, \quad n_y = t_x.$$

This normal is a consistent mathematical convention only; it is not labeled as inward or outward.

5.6 Typed Containers

`BranchCenterline` stores:

- branch name;
- ordered x, y coordinates;
- cumulative arc length in pixels and micrometres;
- tangent vectors;
- normal vectors;
- total length in pixels and micrometres.

`DeviceGeometry` stores:

- device ID;
- calibration;
- branch centerline dictionary;
- upper and lower junction coordinates;
- endpoint mismatch diagnostics.

5.7 Outputs

```
outputs/geometry/asymmetric_loop_h100/centerline_geometry.csv
outputs/geometry/asymmetric_loop_h100/geometry_summary.json
outputs/geometry/asymmetric_loop_h100/device_geometry.npz
outputs/geometry/asymmetric_loop_h100/centerline_geometry.png
```

5.8 Execution

```
.\.venv\Scripts\python.exe -m scripts.build_geometry \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```


6 Device Region Labeling Module

6.1 Files

- `src/geometry/regions.py`
- `scripts/build_device_regions.py`
- `tests/test_device_regions.py`

6.2 Purpose

The region-labeling module creates a device-level integer label map for physical regions. It is independent of a particular droplet frame and is intended as the spatial support for downstream fractional occupancy calculations.

6.3 Label IDs

ID	Region
0	unassigned/background
1	inlet
2	outlet
3	left branch
4	right branch
5	upper junction
6	lower junction

6.4 Construction

The label map is constructed from:

- the binary channel mask;
- original labeled centerlines;
- junction definitions from the device YAML.

Junction boxes are converted from micrometres to pixels using:

$$\text{size}_{px} = \frac{\text{size}_{\mu m}}{\mu m/px}.$$

For the current device:

$$100 \mu m / 4 \mu m/px = 25 \text{ px}.$$

Pixels outside the junction boxes are assigned to the nearest compatible centerline while remaining clipped to the channel mask.

6.5 Validation

The module validates:

- label-map shape equals channel-mask shape;
- valid label IDs only;
- no nonzero labels outside the channel mask;
- nonempty physical regions;
- branch and inlet/outlet connectivity to the appropriate junctions.

6.6 Outputs

data/geometry/asymmetric_loop_h100/region_labels.npy
data/geometry/asymmetric_loop_h100/region_metadata.json
data/geometry/asymmetric_loop_h100/region_overlay.png

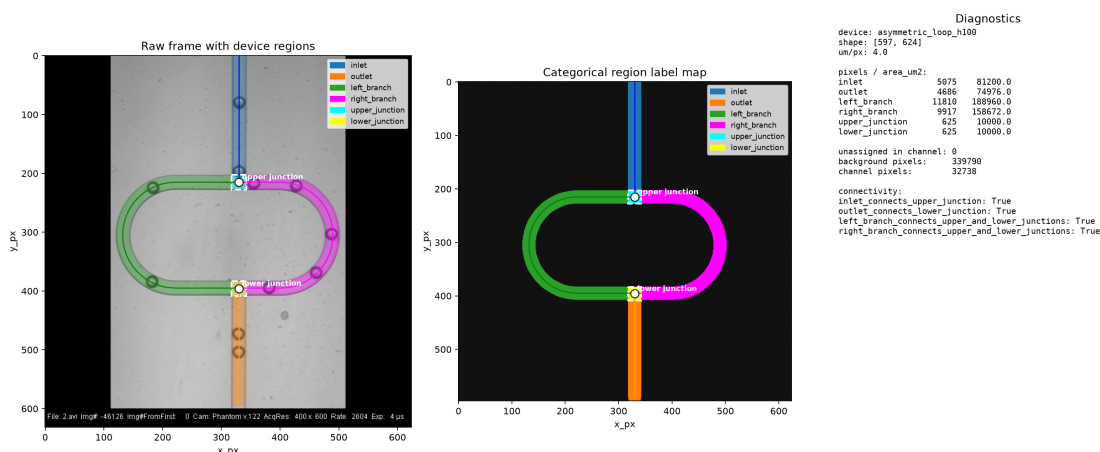


Figure 1: Device-level region overlay produced by `scripts/build_device_regions.py`. The overlay is generated from the precomputed integer region labels, not from visualization-time region inference.

6.7 Execution

```
.\.venv\Scripts\python.exe -m scripts.build_device_regions \  
--experiment configs\experiments\video_2.yml \  
--frame-index 0 \  
--overwrite
```

7 Droplet Fractional Occupancy Module

7.1 Files

- src/occupancy/ellipse.py

- `src/occupancy/calculator.py`
- `src/occupancy/__init__.py`
- `scripts/compute_droplet_occupancy.py`
- `tests/test_droplet_occupancy.py`

7.2 Inputs

The occupancy pipeline reads:

- `outputs/physics/video_2/droplet_occupancy.csv` for downstream use after generation;
- `data/processed/2/tracked_features.csv` as the raw tracking input;
- `data/geometry/asymmetric_loop_h100/region_labels.npy`;
- `data/geometry/asymmetric_loop_h100/region_metadata.json`;
- device calibration from the YAML.

The tracked-feature columns used are:

`frame, track_id, centroid_x, centroid_y, bbox_w, bbox_h`

7.3 Ellipse Footprint

Each droplet footprint is approximated as an axis-aligned ellipse fitted to its tracked bounding box:

$$\left(\frac{x - x_c}{w/2}\right)^2 + \left(\frac{y - y_c}{h/2}\right)^2 \leq 1.$$

Rasterization uses pixel-center coordinates in the same image convention as the region label map:

- x increases with image columns;
- y increases with image rows.

Only the local clipped bounding window around each ellipse is rasterized.

7.4 Raw Occupancy

For physical region k , raw occupancy is:

$$W_{k,\text{raw}} = \frac{\#\{\text{ellipse pixels with label } k\}}{\#\{\text{all rasterized ellipse pixels inside the image}\}}.$$

The denominator is the complete rasterized ellipse area after image-boundary clipping, not only the channel-overlapping part.

The raw physical coverage is:

$$C_{\text{phys}} = \sum_{k=1}^6 W_{k,\text{raw}}.$$

7.5 Normalized Occupancy

For every sample with:

$$C_{\text{phys}} > \epsilon,$$

the normalized six-region occupancy vector is:

$$W_k = \frac{W_{k,\text{raw}}}{C_{\text{phys}}}.$$

The normalized occupancy vector is the canonical representation for downstream ML and physics modules:

$$W = [W_{\text{inlet}} \quad W_{\text{outlet}} \quad W_{\text{left}} \quad W_{\text{right}} \quad W_{\text{upper junction}} \quad W_{\text{lower junction}}].$$

For computable samples:

$$\sum_k W_k = 1.$$

7.6 Diagnostic Low Coverage

The threshold 0.95 is retained as a diagnostic flag:

$$\text{low_physical_coverage} = C_{\text{phys}} < 0.95.$$

Low-coverage samples are still normalized when $C_{\text{phys}} > \epsilon$. They are not called invalid merely because they fall below the diagnostic threshold.

7.7 CSV Columns

Important output columns include:

- diagnostic raw quantities: `w_inlet_raw`, `w_outlet_raw`, `w_left_raw`, `w_right_raw`, `w_upper_junction_raw`, `w_lower_junction_raw`, `physical_coverage_raw`, `low_physical_coverage`;
- pipeline quantities: `occupancy_computable`, `w_inlet`, `w_outlet`, `w_left`, `w_right`, `w_upper_junction`, `w_lower_junction`, `normalized_occupancy_sum`;
- diagnostics: `image_boundary_clipped`, `dominant_region`, `number_active_regions`.

No `w_unassigned` component is created.

7.8 Outputs

`outputs/physics/video_2/droplet_occupancy.csv`
`outputs/physics/video_2/droplet_occupancy_summary.json`
`outputs/physics/video_2/occupancy_diagnostics.png`
`outputs/physics/video_2/occupancy_spot_checks.png`

7.9 Execution

```
.\.venv\Scripts\python.exe -m scripts.compute_droplet_occupancy \
--experiment configs\experiments\video_2.yml \
--overwrite
```

8 Baseline Branch Hydraulics Module

8.1 Files

- src/hydraulics/baseline.py
- src/hydraulics/__init__.py
- scripts/compute_baseline_hydraulics.py
- tests/test_baseline_hydraulics.py

8.2 Purpose

The baseline hydraulics module computes a deterministic frame-level hydraulic state using normalized branch occupancies and a simple equivalent-resistance model. It deliberately excludes inlet, outlet, and junction occupancies from the baseline resistance calculation.

8.3 Effective Branch Occupancy

For frame t :

$$N_{\text{left,eff}}(t) = \sum_{d \in t} W_{\text{left}}(d),$$
$$N_{\text{right,eff}}(t) = \sum_{d \in t} W_{\text{right}}(d).$$

These are continuous effective occupancies, not integer droplet counts.

8.4 Isolated-Droplet Equivalent Length

The dissertation-derived isolated-droplet resistance length is:

$$\ell_d = r_d L_{\text{short}},$$

where:

$$r_d = 0.15.$$

For the current device:

$$L_{\text{short}} = L_{\text{right}} = 1491 \mu\text{m},$$

so:

$$\ell_d = 0.15 \times 1491 = 223.65 \mu\text{m}.$$

8.5 Effective Branch Lengths

$$L_{\text{left,eff}}(t) = L_{\text{left}} + \ell_d N_{\text{left,eff}}(t),$$

$$L_{\text{right,eff}}(t) = L_{\text{right}} + \ell_d N_{\text{right,eff}}(t).$$

8.6 Total Mixture Flow

The hydraulic throughput is the total mixture flow:

$$Q_{\text{mix}} = Q_{\text{continuous}} + Q_{\text{dispersed}}.$$

For video.2:

$$Q_{\text{mix}} = 1950 + 10 = 1960 \text{ } \mu\text{L/hr}.$$

8.7 Parallel Branch Split

Because resistance is proportional to equivalent length and both branches have the same cross-section, the flow split is:

$$Q_{\text{left}} = Q_{\text{mix}} \frac{L_{\text{right,eff}}}{L_{\text{left,eff}} + L_{\text{right,eff}}},$$

$$Q_{\text{right}} = Q_{\text{mix}} \frac{L_{\text{left,eff}}}{L_{\text{left,eff}} + L_{\text{right,eff}}}.$$

The reconstructed total is:

$$Q_{\text{reconstructed}} = Q_{\text{left}} + Q_{\text{right}}.$$

The conservation diagnostic is:

$$\Delta Q = Q_{\text{reconstructed}} - Q_{\text{mix}}.$$

8.8 Superficial Mixture Velocity

The branch velocity columns are total superficial mixture velocities:

$$U = \frac{Q}{WH}.$$

The unit conversion is:

$$1 \text{ } \mu\text{L} = 10^9 \text{ } \mu\text{m}^3, \quad 1 \text{ hr} = 3600 \text{ s}.$$

Thus:

$$U_{\mu\text{m/s}} = \frac{Q_{\mu\text{L/hr}} \cdot 10^9 / 3600}{W_{\mu\text{m}} H_{\mu\text{m}}}.$$

8.9 Canonical ML Features

The canonical hydraulic features intended for the future transformer are only:

`left_velocity_um_s`
`right_velocity_um_s`

These names are kept stable for downstream compatibility, but their documented meaning is:

- `left_velocity_um_s`: total superficial mixture velocity in the left branch;
- `right_velocity_um_s`: total superficial mixture velocity in the right branch.

Flow-rate columns remain in the CSV for diagnostics and conservation checks, but they are not intended as independent ML inputs because they are deterministically related to velocity by the fixed channel cross-section.

8.10 Baseline Assumptions

1. The total volumetric throughput is the sum of continuous- and dispersed-phase input flow rates.
2. The total mixture flow is split between the two branches according to effective hydraulic resistances.
3. Reported branch velocities are superficial mixture velocities based on the full channel cross-sectional area.
4. No phase-specific slip or separate phase velocity is modeled.
5. Branch resistance is proportional to equivalent branch length.
6. Both branches have identical cross-sections.
7. Fluid properties are identical between branches.
8. Droplet resistance contributions are additive.
9. Each isolated droplet contributes 0.15 times the short-branch resistance length.
10. Fractional branch occupancy scales the droplet contribution linearly.
11. Inlet, outlet, upper-junction, and lower-junction occupancies do not contribute to baseline resistance.
12. Droplet–droplet interaction corrections are not modeled.
13. Junction dynamics are reserved for the future learned model.

8.11 Outputs

`outputs/physics/video_2/baseline_hydraulic_state.csv`
`outputs/physics/video_2/baseline_hydraulic_summary.json`
`outputs/physics/video_2/baseline_hydraulic_diagnostics.png`

8.12 Execution

```
.\.venv\Scripts\python.exe -m scripts.compute_baseline_hydraulics \
--experiment configs\experiments\video_2.yml \
--overwrite
```

9 Droplet Branch-Velocity Validation Module

9.1 Files

- scripts/validate_droplet_branch_velocity.py
- tests/test_droplet_branch_velocity_validation.py

9.2 Purpose

This standalone validation analysis compares measured droplet-speed variation against the calculated baseline branch superficial-velocity variation. It does not modify the tracking, occupancy, or hydraulic preprocessing layers.

The analysis is intentionally restricted to branch-interior samples. It excludes inlet, outlet, junction, and branch-transition samples by requiring a normalized branch occupancy of at least 0.95 in exactly one branch:

$$w_{\text{left}} \geq 0.95 \quad \text{xor} \quad w_{\text{right}} \geq 0.95.$$

9.3 Measured Velocity Derivation

For the current `video_2` tracking table, the canonical file is:

```
data/processed/2/tracked_features.csv
```

The table does not contain existing `vx` and `vy` columns, so the validator derives velocity from the configured centroid columns:

```
centroid_x, centroid_y
```

For each track, rows are sorted by frame. Interior samples receive centered finite-difference velocity only when both neighboring observations are exactly one frame apart:

$$v_x(t) = \frac{x(t+1) - x(t-1)}{2}, \quad v_y(t) = \frac{y(t+1) - y(t-1)}{2}.$$

The measured speed is:

$$u_{\text{drop}}(t) = \sqrt{v_x(t)^2 + v_y(t)^2}.$$

Endpoint samples and samples adjacent to tracking gaps are excluded. Track boundaries are never mixed. If future tracking files provide finite `vx/vy` columns, those columns are preferred and the metadata records `tracked_velocity_source = existing_velocity_columns`.

9.4 Unit Interpretation

For the current analysis:

- measured droplet speed units are `px/frame`;
- calculated branch velocity units are $\mu\text{m/s}$;
- `tracked_velocity_source` is `centered_position_difference`.

The validation does not compare absolute velocity magnitudes. Each signal is standardized independently within each selected (`track_id`, `branch`) segment. Therefore, the plots and correlations compare normalized temporal co-variation only. No frame-rate or pixel-scale conversion is required for this normalized comparison.

9.5 Normalization

For every selected track segment, measured speed and calculated branch velocity are normalized separately using sample standard deviation with $ddof = 1$:

$$z_{\text{drop}}(t) = \frac{u_{\text{drop}}(t) - \bar{u}_{\text{drop}}}{\text{std}(u_{\text{drop}})},$$
$$z_{\text{branch}}(t) = \frac{u_{\text{branch}}(t) - \bar{u}_{\text{branch}}}{\text{std}(u_{\text{branch}})}.$$

Tracks with negligible measured-speed or branch-velocity variance are rejected. No smoothing, lag optimization, or physical speed-scale fitting is performed.

9.6 Selection

The intended default minimum branch-interior sample count was 100. The current `video_2` branch-interior segments are shorter than this: left-branch segments top out near 85 samples, while right-branch segments top out near 67 samples. To avoid silently excluding the entire right branch, this analysis uses:

`minimum_branch_interior_samples: 60`

The cutoff is recorded in the summary JSON. Candidate ranking is deterministic: within each branch, candidates are ranked by `std_branch_velocity_um_s` descending, using `track_id` as a tie-breaker. Correlation is not used for selection.

For `video_2`, eligible candidates were imbalanced:

Quantity	Count
Total eligible candidates	3595
Left-branch candidates	1148
Right-branch candidates	2447
Selected candidates	10
Pooled normalized samples	742

9.7 Selected Track Statistics

Track ID	Branch	Samples	Pearson r	Spearman ρ
3463	left	83	0.603518	0.561827
254	left	83	0.756515	0.846252
3127	left	84	0.803099	0.831943
1713	left	84	0.837714	0.859534
3460	left	75	0.527768	0.493072
3461	right	67	0.668621	0.660548
257	right	66	0.767774	0.779150
3465	right	66	0.811301	0.717638
3462	right	67	0.876987	0.876327
1712	right	67	0.458025	0.526406

The pooled descriptive correlations were:

$$r_{\text{Pearson}} = 0.712629, \quad \rho_{\text{Spearman}} = 0.718781.$$

9.8 Outputs

outputs/physics/video_2/droplet_branch_velocity_validation/selected_tracks.csv
outputs/physics/video_2/droplet_branch_velocity_validation/pooled_normalized_samples.csv
outputs/physics/video_2/droplet_branch_velocity_validation/droplet_branch_velocity_summary.json
outputs/physics/video_2/droplet_branch_velocity_validation/droplet_branch_velocity_scatter.png
outputs/physics/video_2/droplet_branch_velocity_validation/track_<track_id>_normalized_velocity.csv

9.9 Execution

```
.\.venv\Scripts\python.exe -m scripts.validate_droplet_branch_velocity \  
--experiment configs\experiments\video_2.yml \  
--overwrite
```

10 Generated Data Products

10.1 Visual Summary

The figures in this section document the main visual artifacts generated during the geometry, occupancy, baseline-hydraulics, and branch-velocity validation steps. They are included directly from their generated repository locations so the document continues to reference the same files consumed during validation.

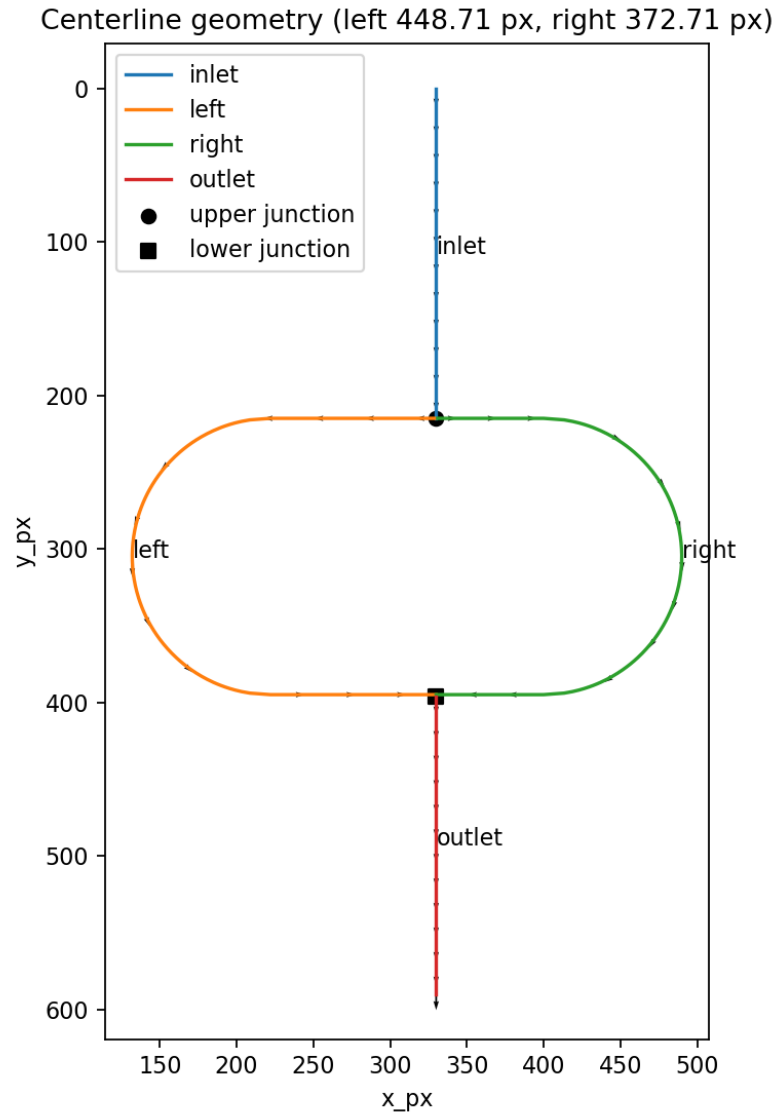


Figure 2: Centerline geometry diagnostic showing the ordered device centerline representation and derived branch structure.

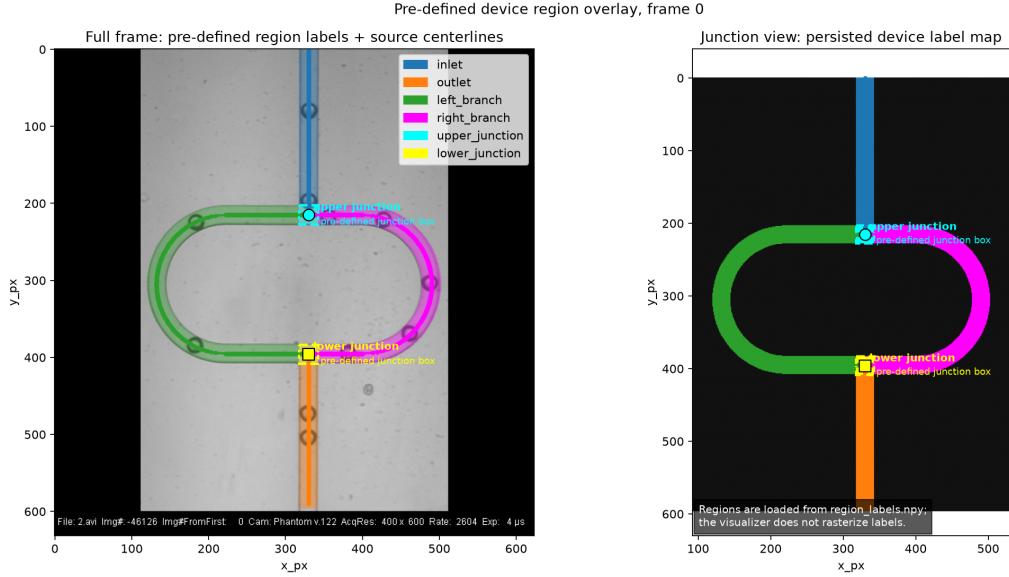


Figure 3: Legacy centerline-region diagnostic for frame 0. This plot was useful during development, but the current occupancy pipeline uses the device-level region labels shown in Figure 1.

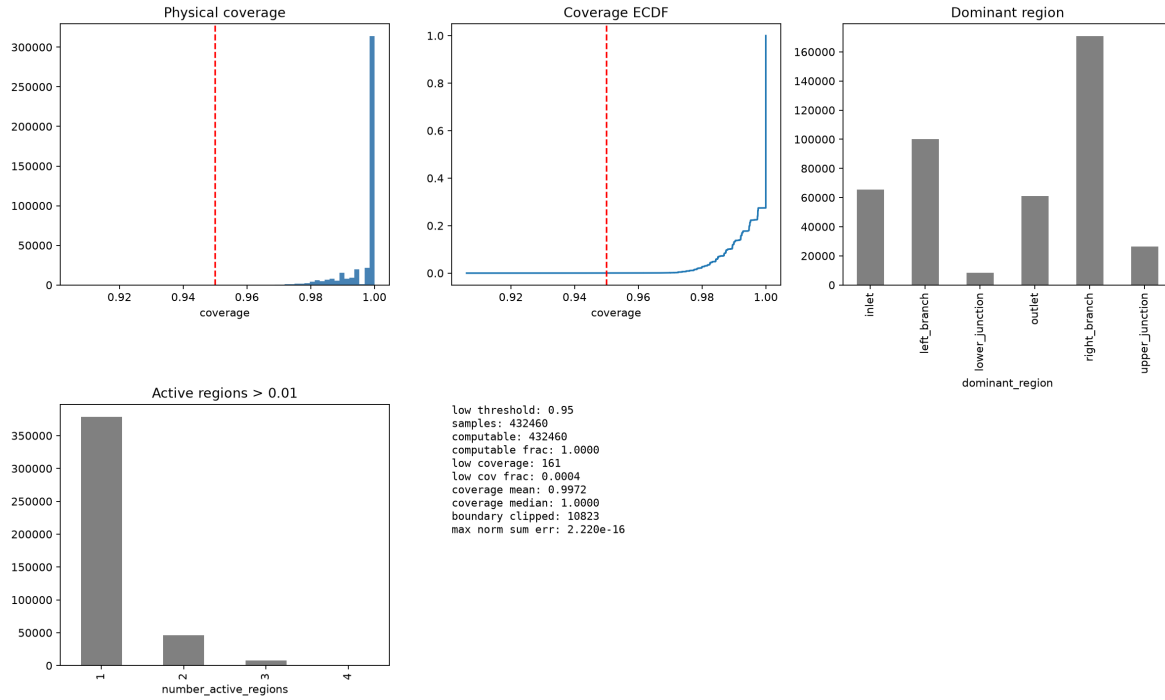


Figure 4: Droplet fractional-occupancy diagnostics summarizing coverage, normalized occupancy behavior, and region assignment statistics.

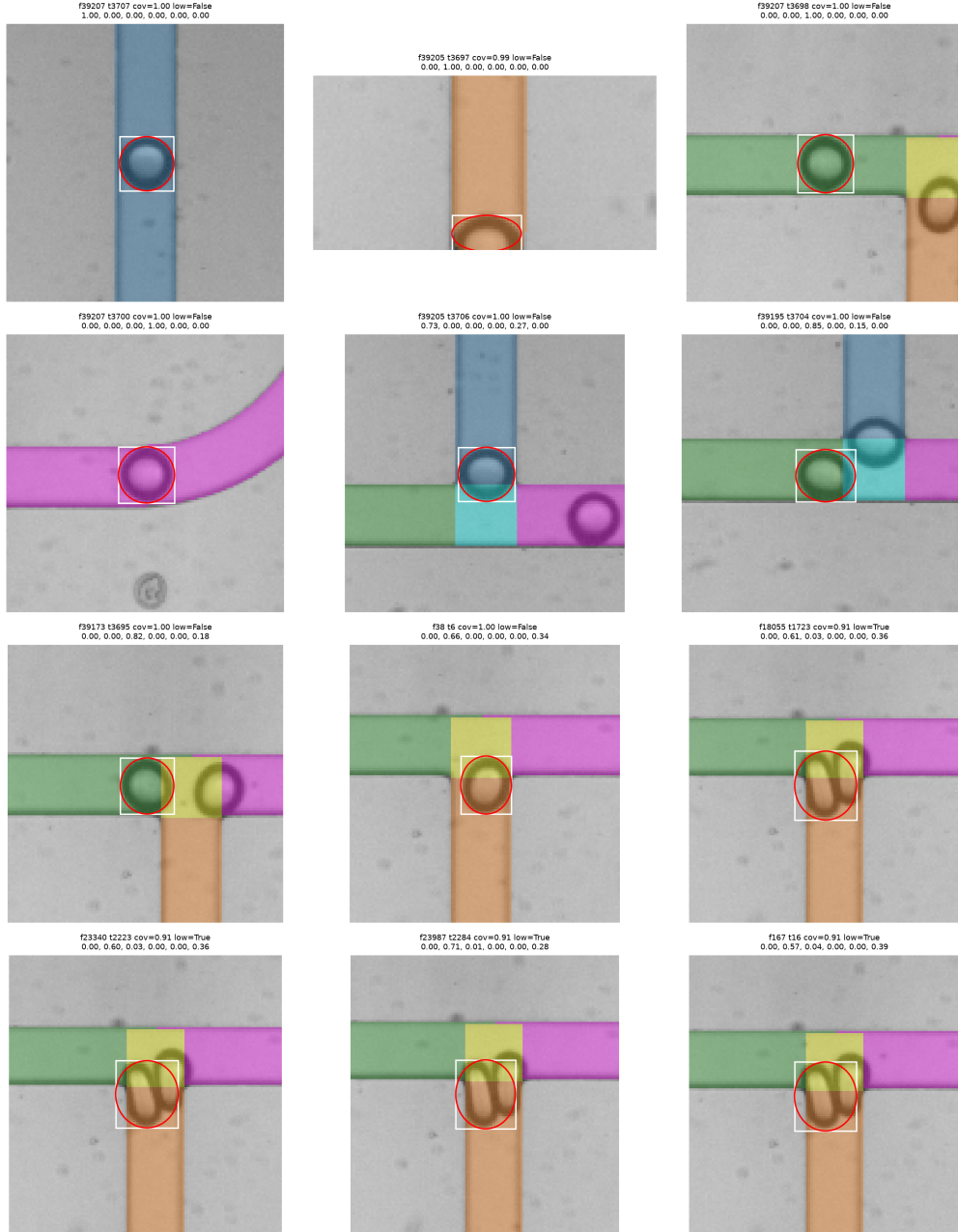


Figure 5: Representative occupancy spot checks used to visually verify ellipse rasterization against the precomputed region labels.

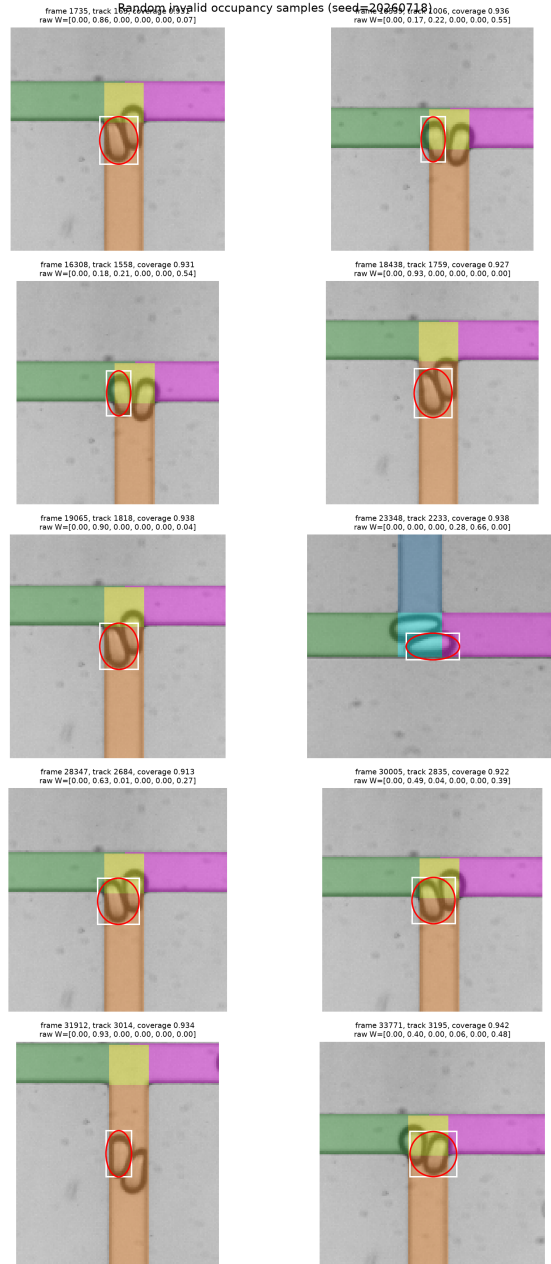


Figure 6: Random sample of ten previously invalid or low-coverage occupancy cases, used for manual inspection of boundary and coverage behavior.

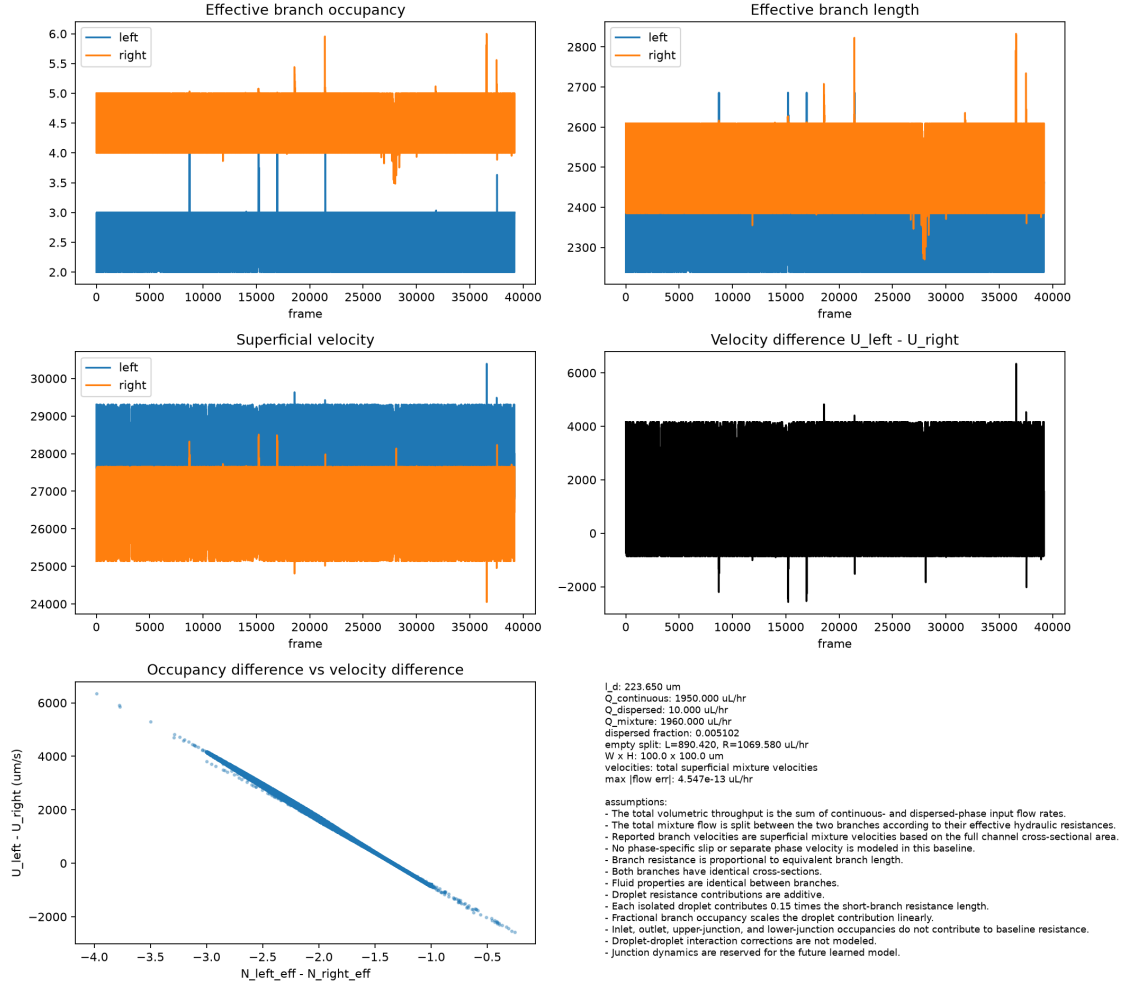


Figure 7: Baseline branch-hydraulics diagnostics showing branch flow-rate and superficial-velocity behavior after normalized branch occupancy is converted into hydraulic resistance.

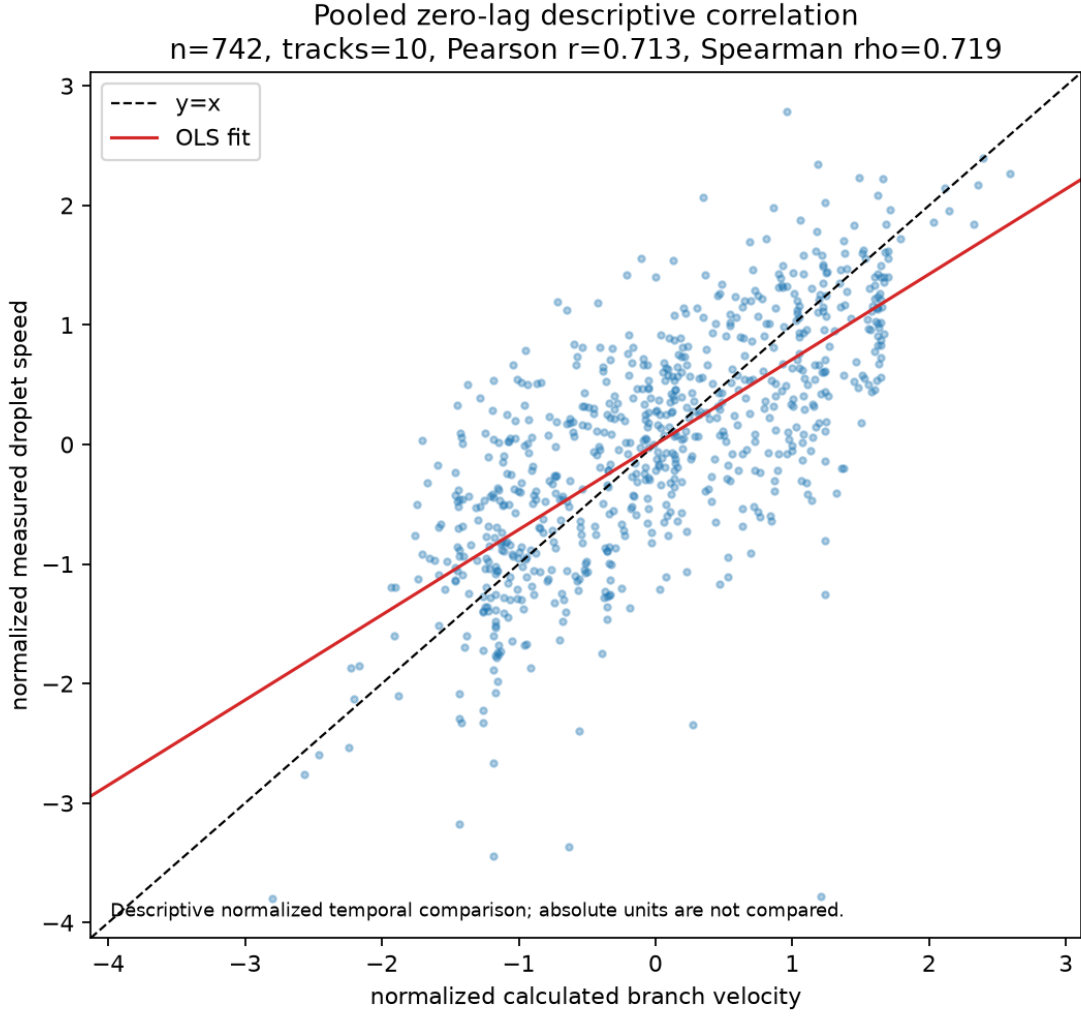


Figure 8: Pooled normalized branch-velocity validation scatter. The x-axis is normalized calculated branch superficial velocity in $\mu\text{m/s}$; the y-axis is normalized measured droplet speed derived in `px/frame`. The plot compares temporal variation after per-track normalization, not absolute velocity magnitudes.

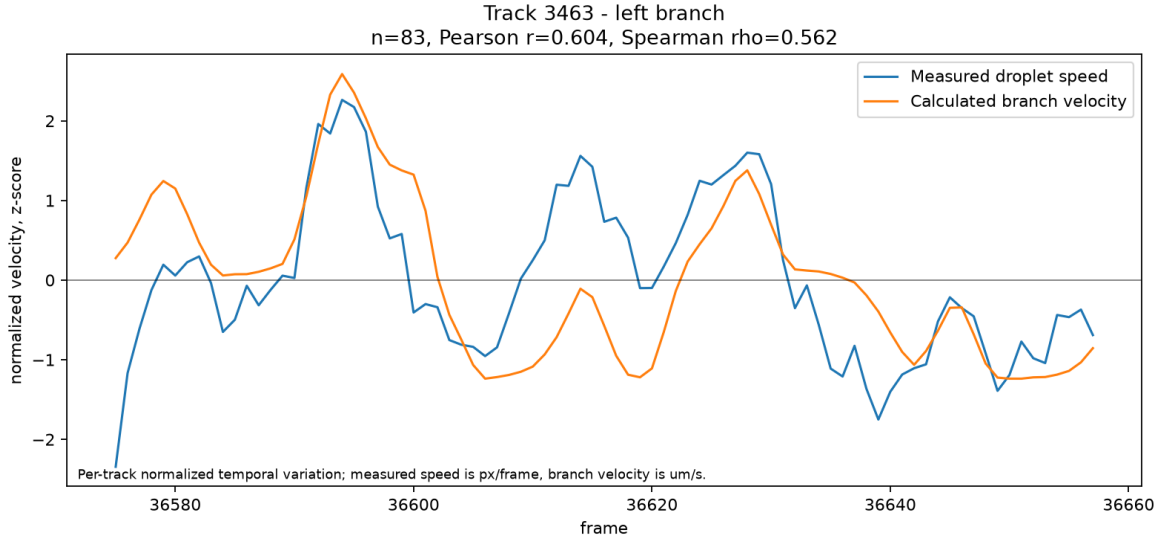


Figure 9: Selected validation track 3463 in the left branch. The measured droplet-speed signal is derived by centered finite difference from `centroid_x/centroid_y`; both signals are shown as per-track z-scores.

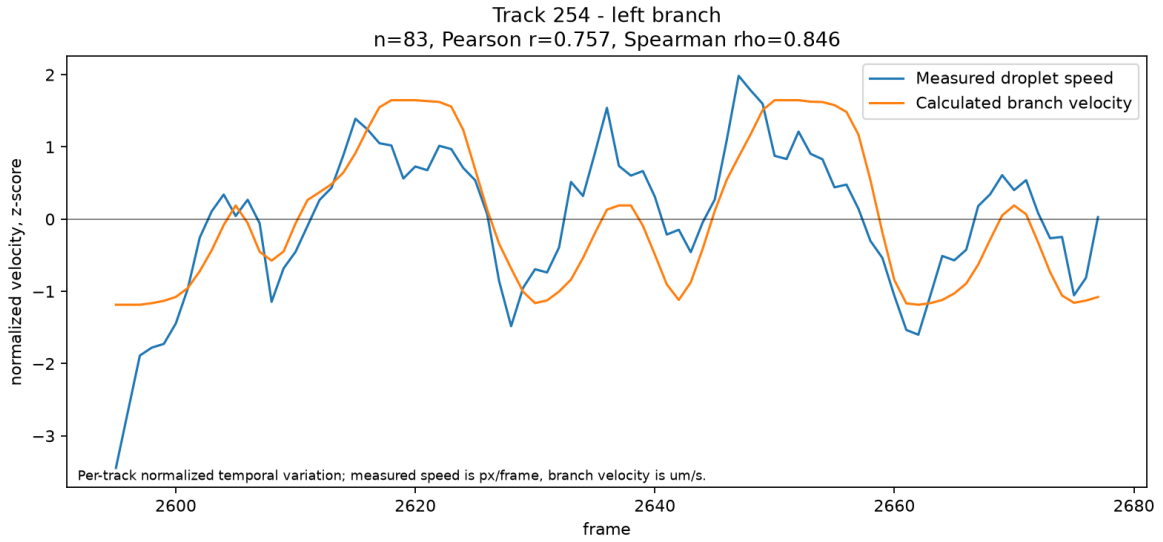


Figure 10: Selected validation track 254 in the left branch. The comparison is zero-lag and unsmoothed.

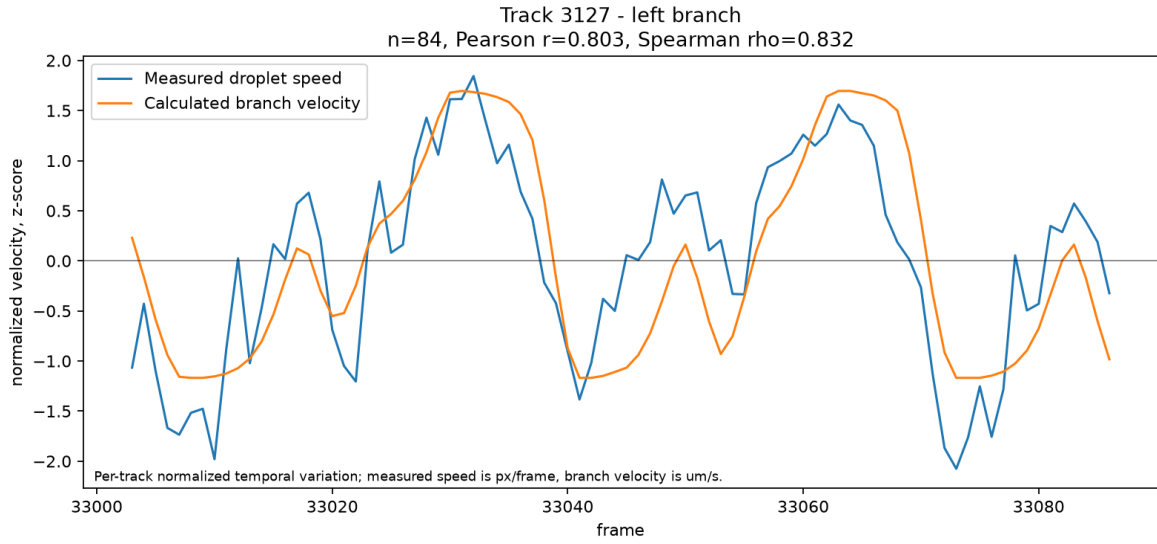


Figure 11: Selected validation track 3127 in the left branch.

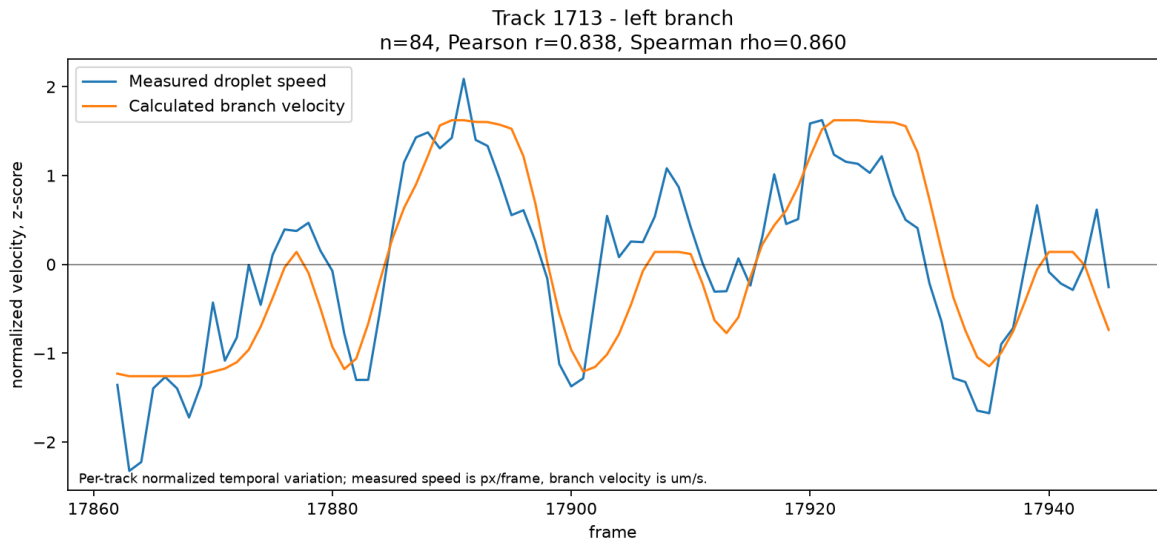


Figure 12: Selected validation track 1713 in the left branch.

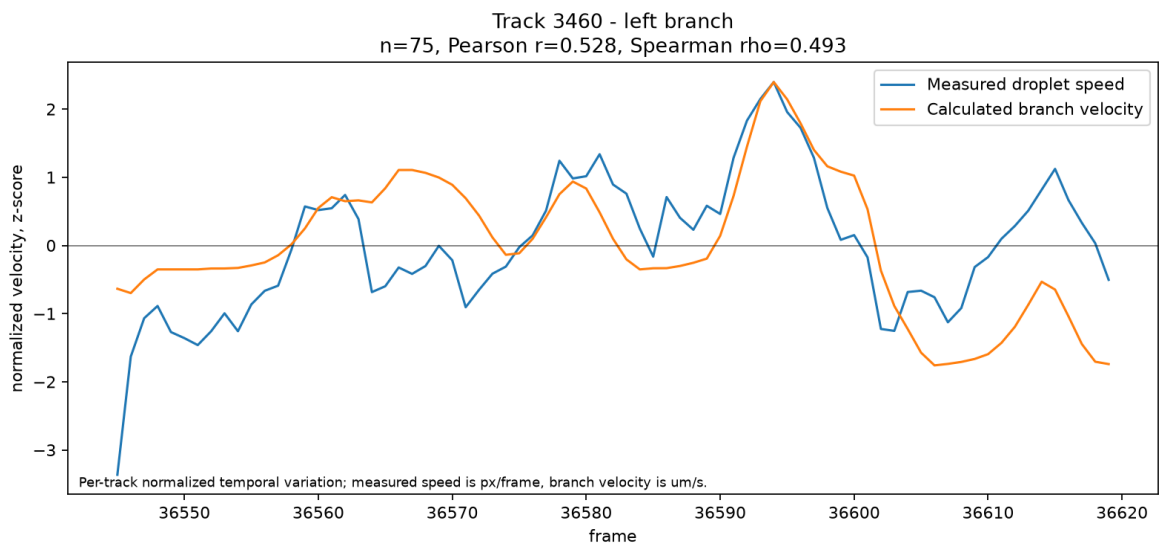


Figure 13: Selected validation track 3460 in the left branch.

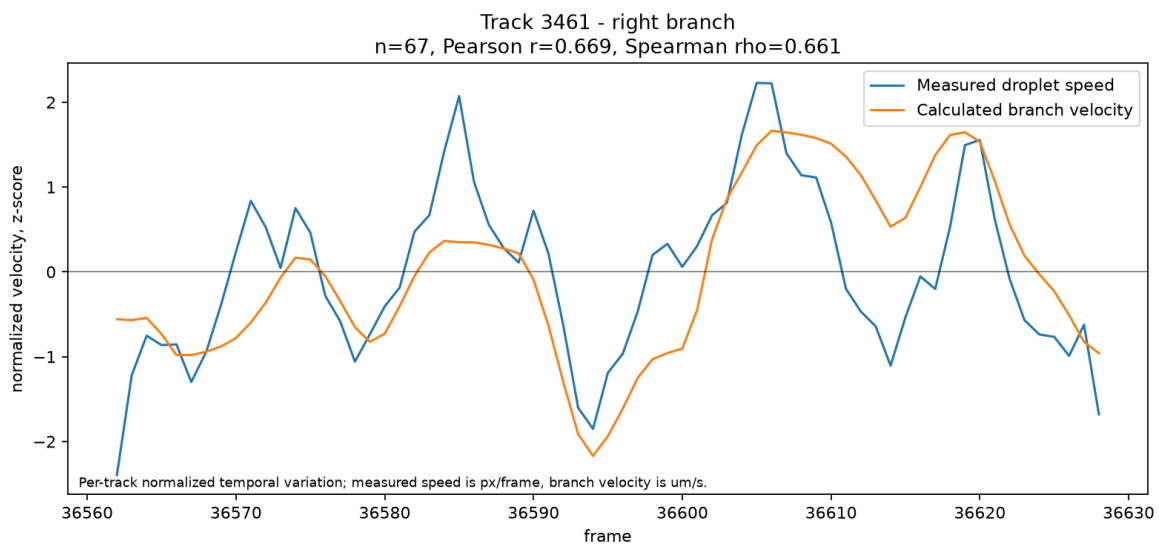


Figure 14: Selected validation track 3461 in the right branch.

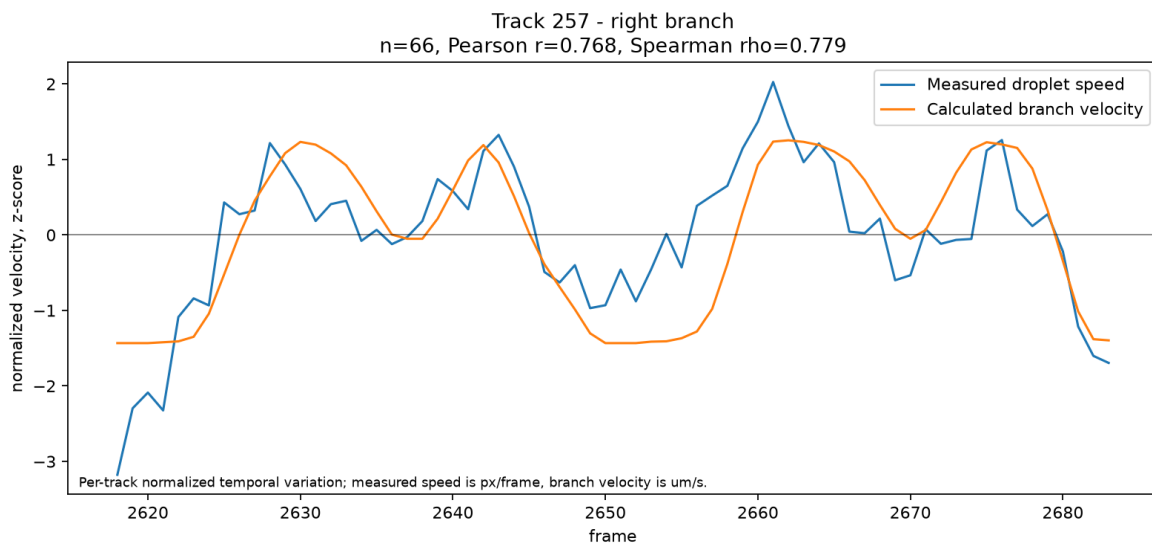


Figure 15: Selected validation track 257 in the right branch.

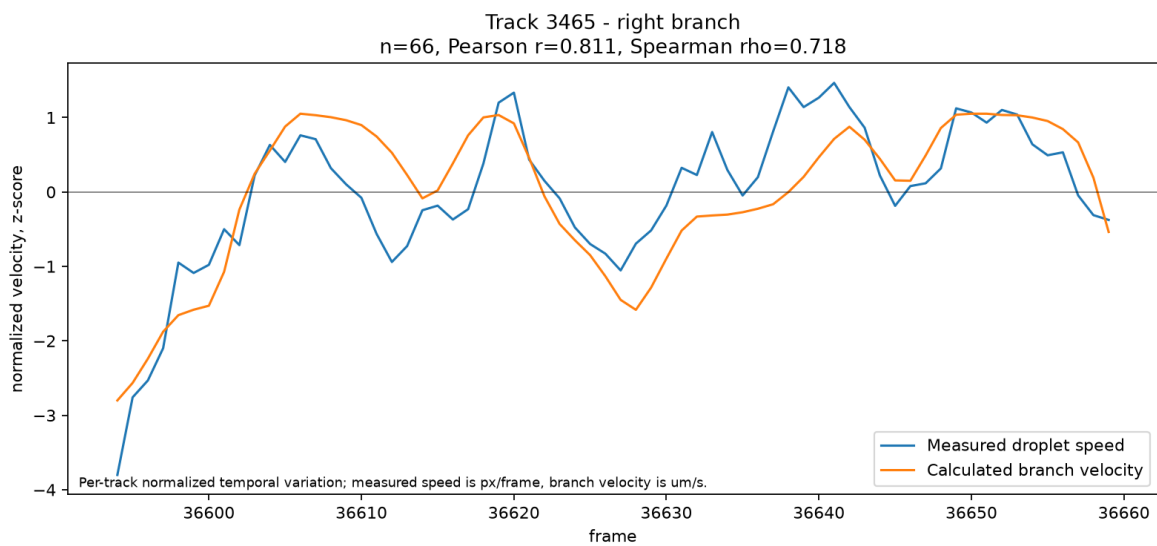


Figure 16: Selected validation track 3465 in the right branch.

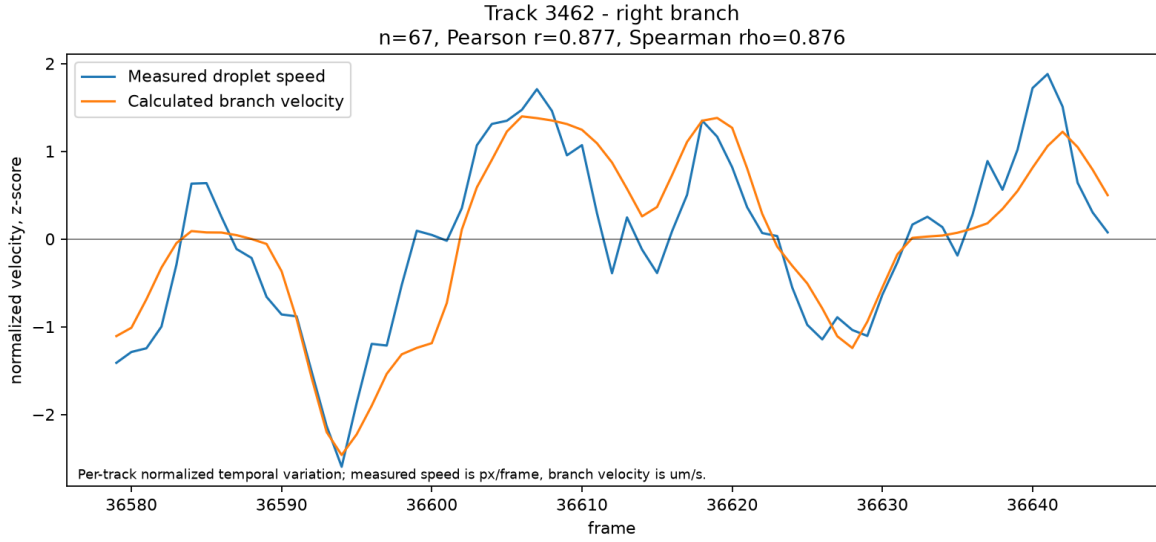


Figure 17: Selected validation track 3462 in the right branch.

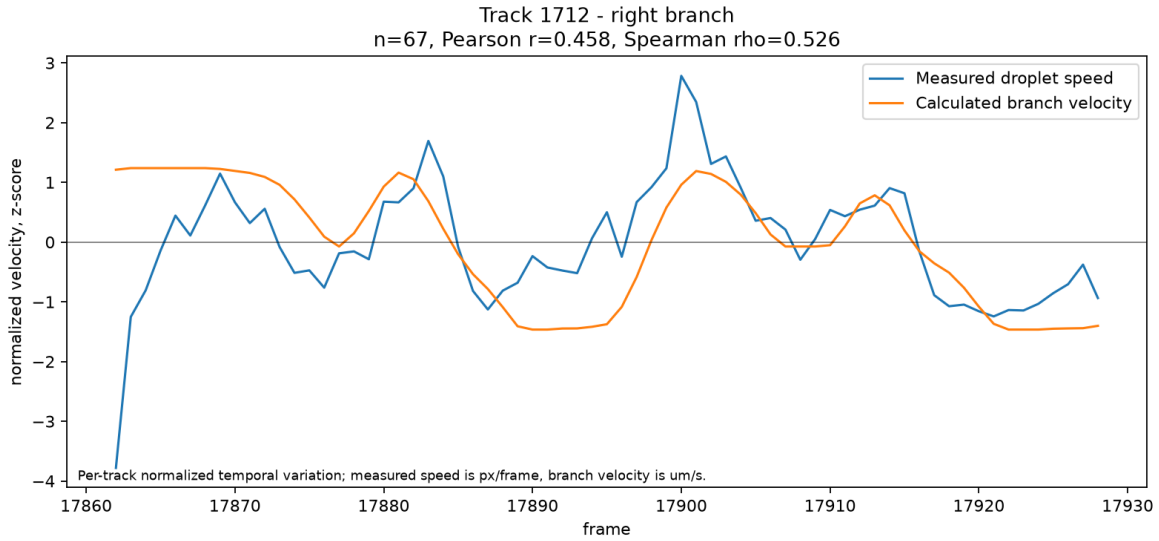


Figure 18: Selected validation track 1712 in the right branch.

10.2 Geometry Products

- outputs/geometry/asymmetric_loop_h100/centerline_geometry.csv
- outputs/geometry/asymmetric_loop_h100/geometry_summary.json
- outputs/geometry/asymmetric_loop_h100/device_geometry.npz
- outputs/geometry/asymmetric_loop_h100/centerline_geometry.png

10.3 Device Region Products

- `data/geometry/asymmetric_loop_h100/region_labels.npy`
- `data/geometry/asymmetric_loop_h100/region_metadata.json`
- `data/geometry/asymmetric_loop_h100/region_overlay.png`

10.4 Occupancy Products

- `outputs/physics/video_2/droplet_occupancy.csv`
- `outputs/physics/video_2/droplet_occupancy_summary.json`
- `outputs/physics/video_2/occupancy_diagnostics.png`
- `outputs/physics/video_2/occupancy_spot_checks.png`

10.5 Hydraulic Products

- `outputs/physics/video_2/baseline_hydraulic_state.csv`
- `outputs/physics/video_2/baseline_hydraulic_summary.json`
- `outputs/physics/video_2/baseline_hydraulic_diagnostics.png`

10.6 Branch-Velocity Validation Products

- `outputs/physics/video_2/droplet_branch_velocity_validation/selected_tracks.csv`
- `outputs/physics/video_2/droplet_branch_velocity_validation/pooled_normalized_samples.csv`
- `outputs/physics/video_2/droplet_branch_velocity_validation/droplet_branch_velocity_summary.json`
- `outputs/physics/video_2/droplet_branch_velocity_validation/droplet_branch_velocity_scatter.png`
- `outputs/physics/video_2/droplet_branch_velocity_validation/track-<track_id>_normalized_velocity.csv`

11 Testing

The test suite currently covers:

- centerline arc length, orientation, tangent, normal, and topology behavior;
- configuration-to-device resolution;
- region label validity and connectivity;
- ellipse rasterization and occupancy normalization;
- low-coverage diagnostics and computability semantics;
- baseline hydraulic equations, unit conversion, flow conservation, and mixture-flow handling;

- centered finite-difference velocity derivation, branch-interior filtering, deterministic candidate selection, correlation calculations, and validation metadata.

Run all tests with:

```
.\.venv\Scripts\python.exe -m pytest tests
```

12 Scope Boundaries

The current implementation does not:

- modify files under `data/processed/2`;
- train or modify a neural network;
- construct transformer training windows;
- model junction resistance;
- model droplet–droplet interactions;
- fit hydraulic parameters from data;
- use segmented droplet masks;
- calculate phase-specific slip or separate phase velocities;
- prove that branch superficial velocity equals droplet speed;
- optimize temporal lag or smooth the validation signals.

13 Recommended Pipeline Order

1. Build centerline geometry:

```
.\.venv\Scripts\python.exe -m scripts.build_geometry \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```

2. Build device region labels:

```
.\.venv\Scripts\python.exe -m scripts.build_device_regions \
  --experiment configs\experiments\video_2.yml \
  --frame-index 0 \
  --overwrite
```

3. Compute droplet occupancy:

```
.\.venv\Scripts\python.exe -m scripts.compute_droplet_occupancy \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```

4. Compute baseline hydraulics:

```
.\.venv\Scripts\python.exe -m scripts.compute_baseline_hydraulics \  
  --experiment configs\experiments\video_2.yml \  
  --overwrite
```

5. Run droplet branch-velocity validation:

```
.\.venv\Scripts\python.exe -m scripts.validate_droplet_branch_velocity \  
  --experiment configs\experiments\video_2.yml \  
  --overwrite
```